Exercícios - Ponteiros e alocação dinâmica

1. Identifique e corrija os erros se existirem:
    a. int *p; *p = 10;
    b. float *v = malloc(5); for (int i = 0; i < 5; i++) v[i] = i * 1.5f;
    c. char *s = malloc(4); strcpy(s, "casa");
    d. int *q = malloc(sizeof(int)); *q = 3; free(q); printf("%d\n", *q);
    e. int *criar() { int x = 5; return &x; }
2. Escreve uma função void ordenar(int *a, int *b) que deixa o menor valor em *a e o maior em *b. Se a ordem já estiver correcta ou se forem iguais, a função não faz nada.
3. Escreve **int *maximo(int *v, int n)**, que devolve o endereço do maior elemento do vector, usando apenas aritmética de ponteiros, sem usar [].
4. Implemente uma função que recebe uma string e um caracter especial e substitua todas as vogais da string pelo caracter recebido. Retorne a string modificada. A função deve ter a seguinte assinatura:
    char * encriptacao(char * s, char c)
5. Escreve a função **int ehPalindromo(const char *s)**, que devolve 1 se a string se lê igual nos dois sentidos, usando dois ponteiros (um no início e outro no fim).
6. Escreve a função **void copiarString(char *destino, const char *origem)** sem usar strcpy. Não te esqueças de copiar o '\0'.
7. Faça um programa que pede ao utilizador um número inteiro N que representa a quantidade de números reais a serem lidos. Utiliza calloc para alocar memória para N valores. Permita que o utilizador introduza os valores. De seguida, percorre o vector dinâmico utilizando ponteiros para encontrar e imprimir o **maior** e o **menor** número armazenado. Liberta a memória no final.
8. Escreva um programa que aloque dinamicamente um vector de inteiros com um tamanho inicial de 5. O programa deve permitir que o utilizador insira vários valores no vector. Caso o vector fique cheio, use realloc para redimensioná-lo, dobrando seu tamanho. No final imprima os números lidos e libere a memória.
9. Escreve int alocar(int **v, int n), que aloca n inteiros e guarda o endereço em *v. Devolve 1 se correu bem e 0 se a alocação falhou.

10. Considere a estrutura abaixo:

```
typedef struct p {
    char nome[50];
    int idade;
} Pessoa;
```

    a. Cria um programa que aloque dinamicamente a estrutura Pessoa, leia os dados, mostre e de seguida libere a memória.
    b. Crie um vector dinâmico de estruturas Pessoa. O programa deve ler a quantidade de pessoas, armazenar os dados e imprimir todas as pessoas cadastradas.